

```
;;;;;;;;;;;;;;  
;; COLLECTIVE KNOW-HOW PROTOTYPE  
;;;;;;;;;;;;;;
```

```
breed [firms firm]  
breed [workers worker]  
breed [products product]  
breed [customers customer]
```

```
undirected-link-breed [coordinations coordination]
```

```
workers-own [
```

```
  skill  
  employer-id
```

```
]
```

```
firms-own [
```

```
  firm-id  
  capability-portfolio  
  capability-maturity  
  capital
```

```
]
```

```
products-own [
```

```
  originating-firm  
  product-type  
  price
```

```
]
```

```
globals [
```

```
  skill-universe  
  product-templates
```

```
]
```

```
to create-workers-for-firms
```

```
  let firm-list sort firms
```

```
  foreach firm-list
```

```
  [  
    my-firm ->
```

```
    create-workers 8  
    [
```

```
      set employer-id [who] of my-firm
```

```
      set skill one-of skill-universe
```

```
      set shape "person"
```

```
      set size 1
```

```
      move-near-firm my-firm
```

```
      color-by-skill
```

```
    ]
```

```

]
end

to color-by-skill

  if skill = "marketing"
  [ set color yellow ]

  if skill = "payments"
  [ set color cyan ]

  if skill = "customer-service"
  [ set color green ]

  if skill = "data-analytics"
  [ set color orange ]

  if skill = "cloud-computing"
  [ set color violet ]

  if skill = "ai-recommendation"
  [ set color red ]

  set label skill

end

to move-near-firm [my-firm]

  let r (2 + random-float 2)

  let theta random-float 360

  setxy
    ([xcor] of my-firm + r * cos theta)
    ([ycor] of my-firm + r * sin theta)

end

to create-coordination-links

  ask firms
  [
    let my-workers
      workers with
        [employer-id = [who] of myself]

    ;; seleccionar tres workers

    if count my-workers >= 3
    [
      let w1 item 0 sort my-workers
      let w2 item 1 sort my-workers
      let w3 item 2 sort my-workers

      ask w1
      [
        create-coordination-with w2
        [
          set color black
          set thickness 0.4
        ]

        create-coordination-with w3

```

```

    [
      set color black
      set thickness 0.4
    ]
  ]

ask w2
[
  create-coordination-with w3
  [
    set color black
    set thickness 0.4
  ]
]

]

]

end

to build-capabilities

ask firms
[
  set capability-portfolio []

  let available-skills
    firm-skills self

  let ck
    collective-knowhow-score self

  ;; Commercial Capability

  if ck >= 1
    and
    is-subset?
    ["marketing"
     "payments"
     "customer-service"]
    available-skills
  [
    if not member?
      "commercial-capability"
      capability-portfolio
    [
      set capability-portfolio
        lput
        "commercial-capability"
        capability-portfolio
    ]
  ]

  ;; Digital Capability

  if ck >= 2
    and
    is-subset?
    ["data-analytics"
     "cloud-computing"]
    available-skills
  [
    if not member?
      "digital-capability"
      capability-portfolio
  ]
]

```

```

    [
      set capability-portfolio
        lput
          "digital-capability"
          capability-portfolio
    ]
  ]

;; AI Capability

if ck >= 3
  and
  is-subset?
    ["data-analytics"
     "cloud-computing"
     "ai-recommendation"]
    available-skills
  [
    if not member?
      "ai-capability"
      capability-portfolio
    [
      set capability-portfolio
        lput
          "ai-capability"
          capability-portfolio
    ]
  ]
]

end

to market-dynamics
  customer-shopping
  pay-operational-costs
  check-bankruptcies
end

to customer-shopping
  ask customers [
    ;; Los clientes se mueven aleatoriamente buscando productos
    rt random 40 - 20
    fd 0.5

    ;; Buscar productos cercanos en un radio de 3 parches
    let nearby-products products in-radius 6

    if any? nearby-products [
      ;; Elige el producto de la firma con mayor CK score
      let chosen-product max-one-of nearby-products [
        collective-knowhow-score firm originating-firm
      ]

      let vendor firm [originating-firm] of chosen-product

      if vendor != nobody [
        ;; Transferencia de capital por venta
        ask vendor [
          set capital capital + [price] of chosen-product
        ]

        ;; Efecto visual: el cliente salta brevemente a la posición del producto consumido
        move-to chosen-product
        setxy random-xcor random-ycor ;; El cliente reaparece en otro punto del mercado
      ]
    ]
  ]
]

```

```

]
end

to pay-operational-costs
  ask firms [
    let my-workers count workers with [employer-id = [who] of myself]
    let my-products count products with [originating-firm = [who] of myself]

    ;; Costo operativo: $1 por trabajador + $0.5 por producto activo por tick
    let payroll-cost my-workers * 0.2
    let maintenance-cost my-products * 0.1

    set capital capital - (payroll-cost + maintenance-cost)
  ]
end

to entry-of-new-firms
  ;; Si quedan menos de 3 firmas, hay una probabilidad de que nazca una startup
  if (count firms < 3) and (random-float 1.0 < 0.05) [
    create-firms 1 [
      set shape "circle"
      set size 4
      set color blue
      setxy random-xcor random-ycor
      set firm-id who
      set label (word "F" who)
      set capability-portfolio []
      set capital 300

      let my-firm self

      ;; Usamos hatch-workers porque 'self' es una firm (turtle)
      hatch-workers 8 [
        set employer-id [who] of my-firm
        set skill one-of skill-universe
        set shape "person"
        set size 1
        move-near-firm my-firm
        color-by-skill
      ]
    ]
  ]
end

to check-bankruptcies
  ask firms with [capital <= 0] [
    let dead-firm-id who

    ;; Eliminar trabajadores de la firma en quiebra
    ask workers with [employer-id = dead-firm-id] [ die ]

    ;; Eliminar productos de la firma en quiebra
    ask products with [originating-firm = dead-firm-id] [ die ]

    ;; La firma desaparece
    die
  ]
end

to create-customers-demo

  create-customers 20
  [
    set shape "person"

```

```

    set color red

    set size 1

    setxy random-xcor random-ycor
  ]
end

to grow-coordination-network
  ask firms
  [
    let firm-workers
      workers with
        [employer-id = [who] of myself]

    let firm-links
      coordinations with
        [
          [employer-id] of end1 = [who] of myself
          and
          [employer-id] of end2 = [who] of myself
        ]

    ;; =====
    ;; FASE 1: NUCLEACIÓN
    ;; =====

    ifelse not any? firm-links
    [
      let w1 one-of firm-workers

      let candidates
        other firm-workers
        with
          [skill != [skill] of w1]

      if any? candidates
      [
        let w2 one-of candidates

        ask w1
        [
          create-coordination-with w2
          [
            set color black
            set thickness 0.2
          ]
        ]
      ]
    ]

    [
      let connected-workers
        (turtle-set
          [end1] of firm-links
          [end2] of firm-links)

      let isolated-workers
        firm-workers with

```

```

[not member? self connected-workers]

;; =====
;; FASE 2: EXPANSIÓN
;; =====

ifelse any? isolated-workers
[
  let newcomer
    one-of isolated-workers

  let anchor
    one-of connected-workers

  if [skill] of newcomer !=
    [skill] of anchor
  [
    ask newcomer
    [
      create-coordination-with anchor
      [
        set color black
        set thickness 0.2
      ]
    ]
  ]
]

[
  ;; =====
  ;; FASE 3: CIERRE TRIÁDICO
  ;; =====

  let eligible-workers
    connected-workers
    with [any? link-neighbors]

  if any? eligible-workers
  [
    let w1 one-of eligible-workers

    let first-neighbors
      [link-neighbors] of w1

    let second-neighbors
      turtle-set
      [link-neighbors] of first-neighbors

    let candidates
      second-neighbors
      with
      [
        employer-id = [who] of myself
        and
        self != w1
        and
        not link-neighbor? w1
      ]

    if any? candidates
    [

```

```

    let w2 one-of candidates

    if [skill] of w1 !=
      [skill] of w2
    [
      ask w1
      [
        create-coordination-with w2
        [
          set color black
          set thickness 0.2
        ]
      ]
    ]
  ]
]
]
]
end

to create-emergent-products
  let products-to-create []

  ask firms
  [
    let current-firm-id who

    let available-capabilities
      capability-portfolio

    let firm-x xcor
    let firm-y ycor

    foreach product-templates
    [
      template ->

      let product-name
        item 0 template

      let required-capabilities
        item 1 template

      if is-subset?
        required-capabilities
        available-capabilities
      [
        if not any? products with
        [
          originating-firm = current-firm-id
          and
          product-type = product-name
        ]
        [
          set products-to-create

```

```

        lput
        (list
         current-firm-id
         product-name
         firm-x
         firm-y)
        products-to-create
    ]
]
]
]
foreach products-to-create
[
    pdata ->

    create-products 1
    [
        set shape "box"
        set color green
        set size 2

        set originating-firm
            item 0 pdata

        set product-type
            item 1 pdata

        set label
            item 1 pdata

        set price 15

        setxy
            item 2 pdata
            item 3 pdata

        fd 4
    ]
]
end

to-report is-subset? [small-set big-set]

    report empty?
        filter
            [x -> not member? x big-set]
        small-set
end

to-report firm-skills [firm-agent]

    report remove-duplicates
        [skill] of workers
        with [employer-id = [who] of firm-agent]
end

```

```
to-report firm-has-diverse-triangle? [firm-agent]
```

```
  let my-workers
    workers with
      [employer-id = [who] of firm-agent]

  let worker-list sort my-workers

  foreach worker-list
  [
    w1 ->

    foreach worker-list
    [
      w2 ->

        foreach worker-list
        [
          w3 ->

            if (w1 != w2) and
              (w1 != w3) and
              (w2 != w3)
            [
              if ([link-neighbor? w2] of w1) and
                ([link-neighbor? w3] of w1) and
                ([link-neighbor? w3] of w2)
              [
                let skills
                  remove-duplicates
                    (list
                     [skill] of w1
                     [skill] of w2
                     [skill] of w3)

                if length skills = 3
                [
                  report true
                ]
              ]
            ]
          ]
        ]
      ]
    ]
  ]

  report false
```

```
end
```

```
to-report triangles-in-firm [firm-agent]
```

```
  let my-workers
    workers with
      [employer-id = [who] of firm-agent]

  let count-triangles 0

  ask my-workers
  [
    let my-neighbors
      link-neighbors with
        [member? self my-workers]

    if count my-neighbors >= 2
```

```

    [
      set count-triangles
        count-triangles + 1
    ]
  ]

  report count-triangles
end

to-report firms-with-triangles

  report count firms
    with
      [firm-has-diverse-triangle? self]
end

to-report collective-knowhow-score [firm-agent]

  let my-workers
    workers with
      [employer-id = [who] of firm-agent]

  let score 0

  let worker-list sort my-workers

  foreach worker-list
  [
    w1 ->

    foreach worker-list
    [
      w2 ->

      foreach worker-list
      [
        w3 ->

        if (w1 != w2) and
          (w1 != w3) and
          (w2 != w3)
        [

          if ([link-neighbor? w2] of w1) and
            ([link-neighbor? w3] of w1) and
            ([link-neighbor? w3] of w2)
          [

            let skills
              remove-duplicates
              (list
                [skill] of w1
                [skill] of w2
                [skill] of w3)

            if length skills = 3
            [
              set score score + 1
            ]
          ]
        ]
      ]
    ]
  ]
end

```

```

report score / 6
end

to adapt-workforce
ask firms [
  ;; Evaluación por firma: si CK es alto y hay espacio en plantilla (< 12)
  if (collective-knowhow-score self > 30) and (count workers with [employer-id = [who] of myself] < 12) [
    hiring-process
  ]
]
end

to hiring-process ;; Se ejecuta dentro del contexto de la firma (turtle)
let my-firm self
let current-skills firm-skills my-firm
let missing-skills filter [s -> not member? s current-skills] skill-universe

if not empty? missing-skills [
  let needed-skill one-of missing-skills

  ;; Usamos 'hatch-workers' porque el agente ejecutor es una 'firm'
  hatch-workers 1 [
    set employer-id [who] of my-firm
    set skill needed-skill
    set shape "person"
    set size 1
    move-near-firm my-firm
    color-by-skill
  ]
]
end

to update-firm-visualization
ask firms
[
  let ck collective-knowhow-score self
  set size 3 + (ck / 10) ;; Escalar tamaño

  ;; Sintaxis corregida: (word arg1 arg2 arg3 arg4 arg5 arg6)
  set label (word "F" who " | CK=" (precision ck 0) " | $" (precision capital 0))
]
end

to-report commercial-capability-firms

report count firms with
[member?
"commercial-capability"
capability-portfolio]

end

to-report digital-capability-firms

report count firms with
[member?
"digital-capability"
capability-portfolio]

end

to-report ai-capability-firms

report count firms with
[member?
"ai-capability"

```

```

        capability-portfolio]
end

;Setup

to setup
    clear-all

    ask patches
    [
        set pcolor white
    ]

    set product-templates
    [
        ["BasicCommerce"
         ["commercial-capability"]]

        ["Marketplace"
         ["commercial-capability"]]

        ["CloudService"
         ["digital-capability"]]

        ["AIService"
         ["ai-capability"]]
    ]

    set skill-universe
    [
        "marketing"
        "payments"
        "customer-service"
        "data-analytics"
        "cloud-computing"
        "ai-recommendation"
    ]

    create-firms 5
    [
        set shape "circle"
        set size 4
        set color blue

        setxy random-xcor random-ycor

        set firm-id who
        set label word "F" who
        set capability-portfolio []
        set capital 500
    ]

    create-workers-for-firms

;create-coordination-links

create-customers-demo

    reset-ticks

```

```
end
; GO
to go

  grow-coordination-network

  build-capabilities

  market-dynamics ;; <--- NUEVO: Interacción de compra, costos y quiebras

  update-firm-visualization

  create-emergent-products

  adapt-workforce

  entry-of-new-firms

  tick

end
```